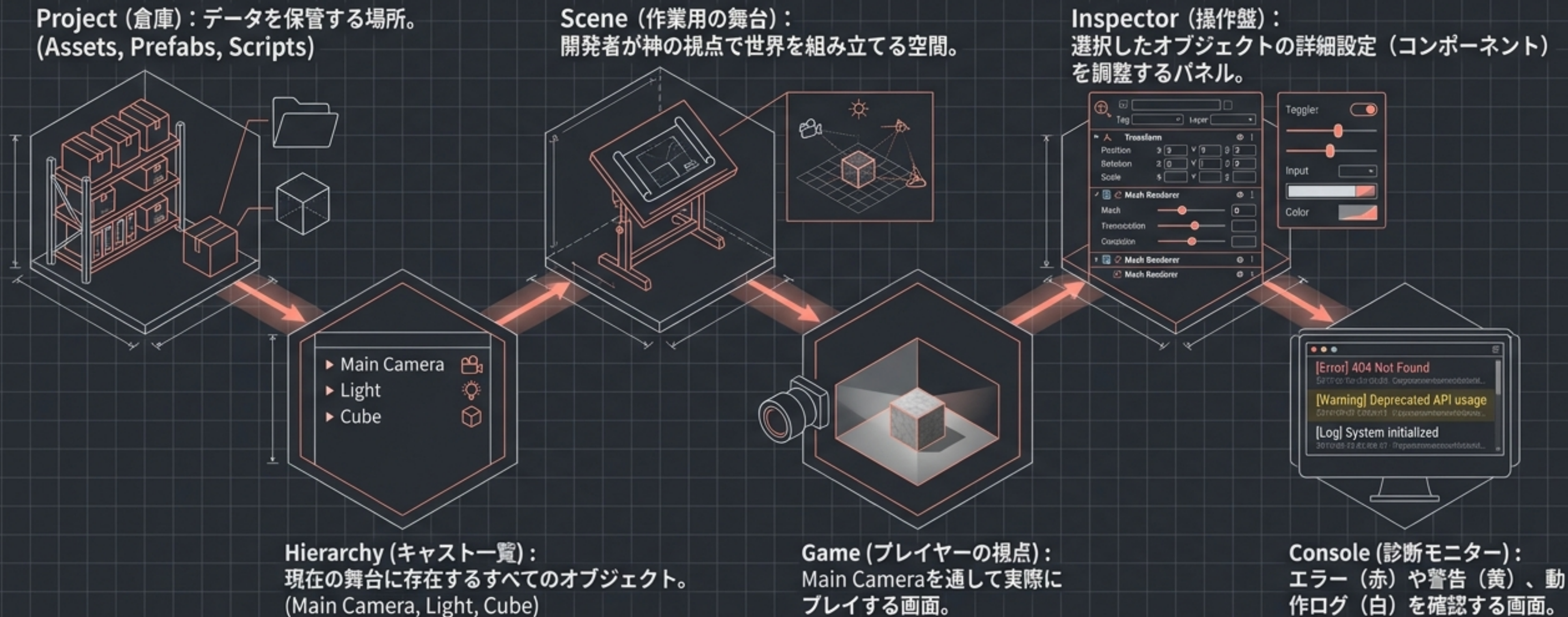


ゼロから創る、Unity世界の解体新書

点と点が繋がる、ゲーム開発のシステム思考アーキテクチャ



開発のコックピット：情報とアセットの情報の流れ



Caution: Playモード中の変更は、停止するとすべてリセットされます！変更は必ず停止後に行うこと。
(レイアウトが崩れた場合は Window > Layouts > Reset All Layouts)

世界に命を吹き込む「神経系」：Input System

物理デバイス
(Device)

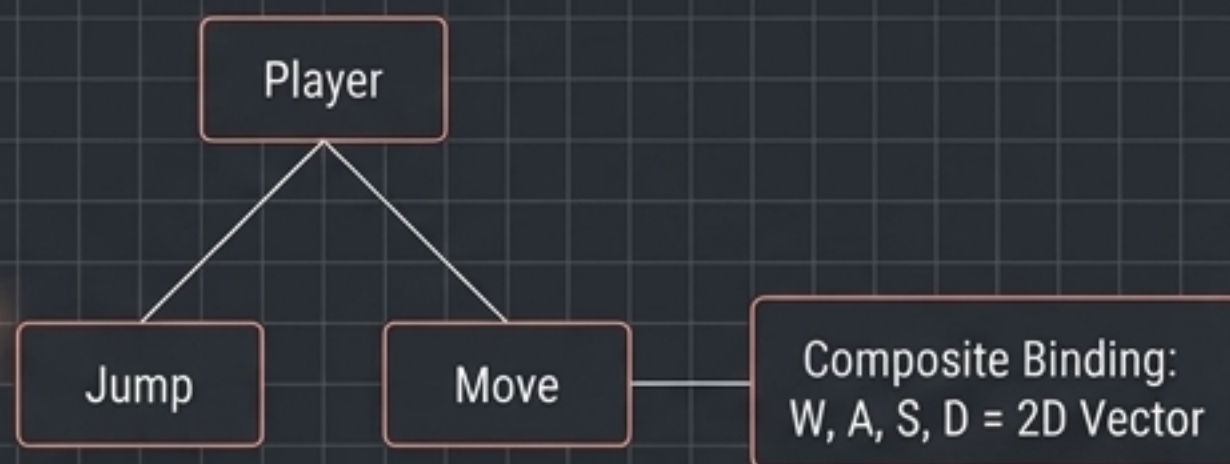


InputActions
(設定ファイル)

Binding
(割り当て)

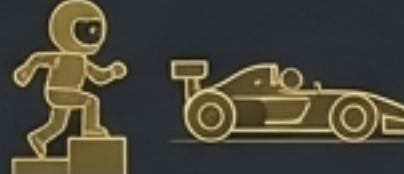


Script
(C# イベント)



```
1
2 // Actionの3つの状態 (コールバック)
3 action.Player.Move.started += OnMove; // 入力開始
4 action.Player.Move.performed += OnMove; // 入力中
5 action.Player.Move.canceled += OnMove; // 入力終了
```

2つの移動哲学：Transform vs Rigidbody

特徴	Transform (座標の直接操作)	Rigidbody (物理演算ベース)
概念 (Concept)	神の視点による「瞬間移動（テレポート）」 	現実世界のような「力の加算（プッシュ）」 
更新メソッド	Update() + Time.deltaTime (フレーム依存を回避) <pre>Update() { Time.deltaTime (フレーム }</pre>	FixedUpdate() (一定間隔での物理計算) <pre>FixedUpdate() { (一定間隔での物理計算) }</pre>
主なコード	<pre>transform.position += movement;</pre>	<pre>rb.AddForce() または rb.velocity = movement;</pre>
重力・衝突	無視される (壁をすり抜ける) 	自動的に計算される (重力で落ち、壁で止まる) 
最適ジャンル	2Dの単純移動、UI、 トップダウンゲーム 	アクション、プラットフォーマー、 レースゲーム 

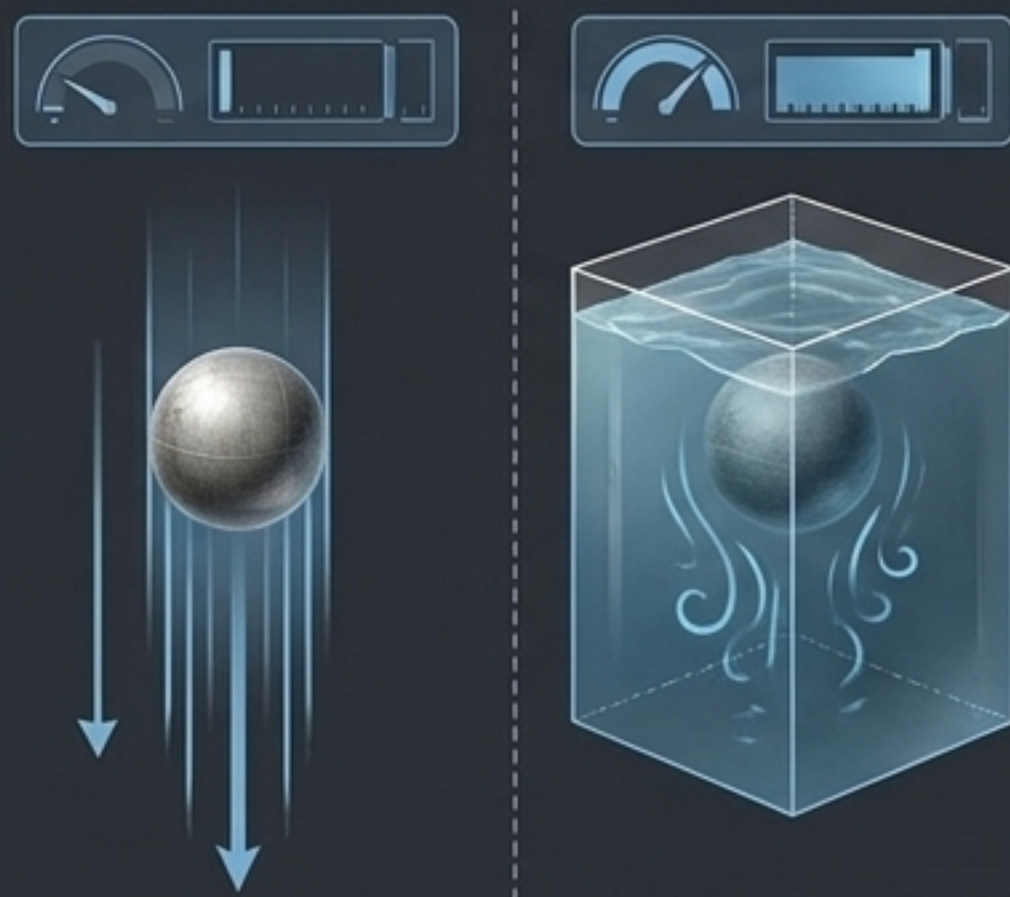
物理演算のリアルを追求する：Rigidbodyの3大要素

Mass (質量)



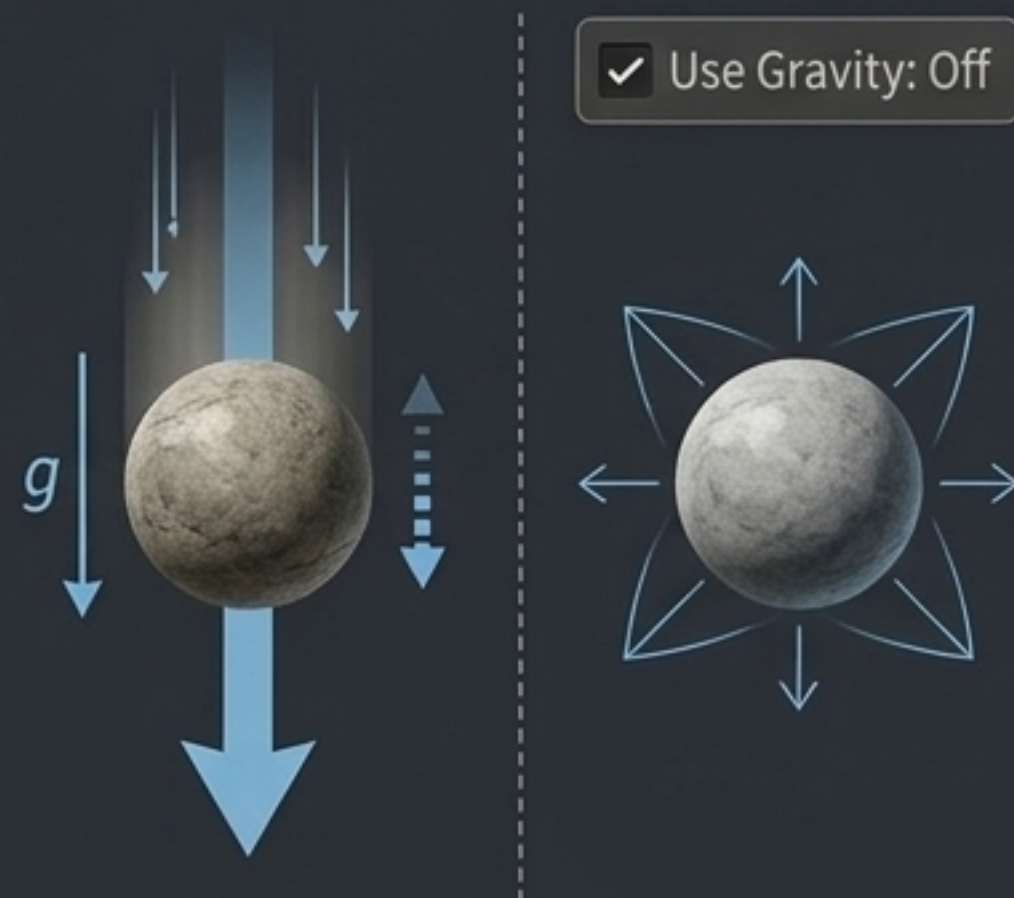
Massを変更すると、衝突時の「押し負け・押し勝ち」が変わる。
(例：Mass 0.5 は吹き飛びやすく、Mass 5 は動かない)

Drag (空気抵抗)



Dragを上げると、移動が失速しやすくなる。氷の上か、水の中かの違いを生む。

Use Gravity (重力の有無)



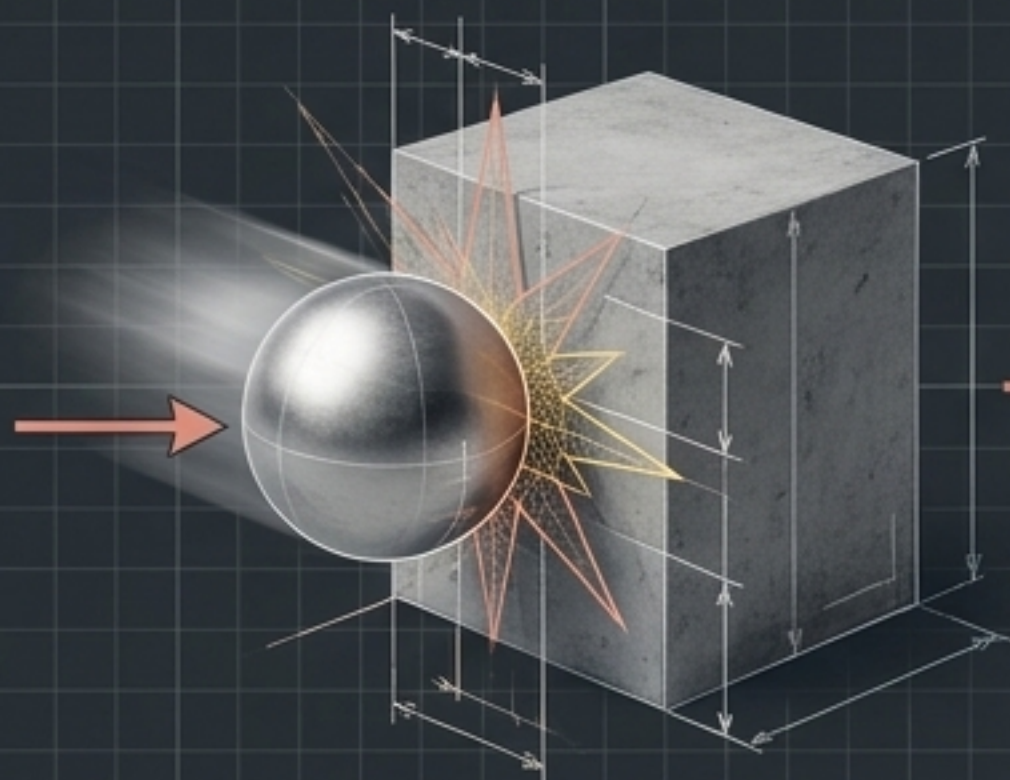
チェックを外すと、宇宙空間やトップダウン視点のように重力を無視して動ける。

衝突というドラマ：Collisionのライフサイクル

Time = 0: 衝突の瞬間

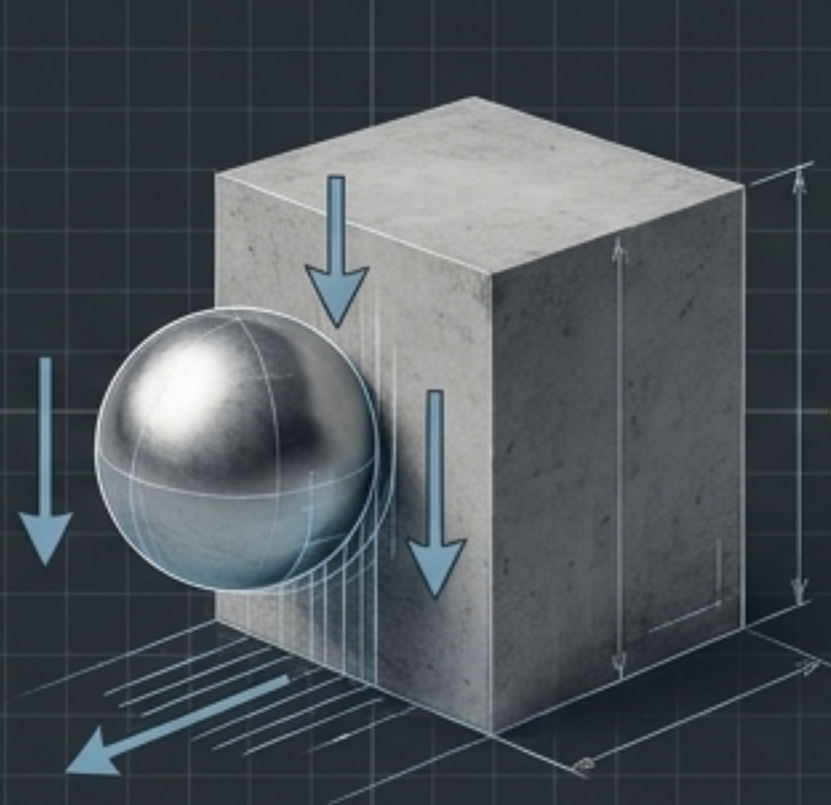
Time = 1: 接触中

Time = 2: 離脱の瞬間



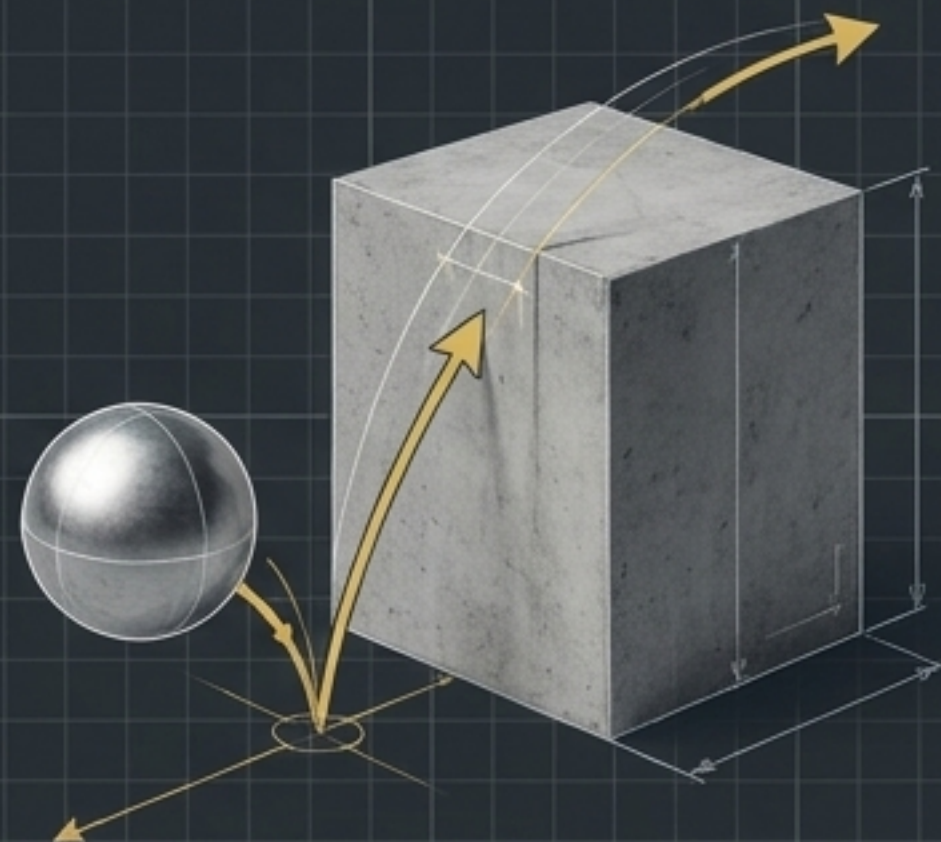
OnCollisionEnter (または
OnCollisionEnter2D)

用途: ダメージ判定、火花のエフェクト
(ContactPointで位置取得)、着地の
効果音。



OnCollisionStay

用途: ダメージ床（溶岩）にいる間の
継続ダメージ、ベルトコンベアの移動。

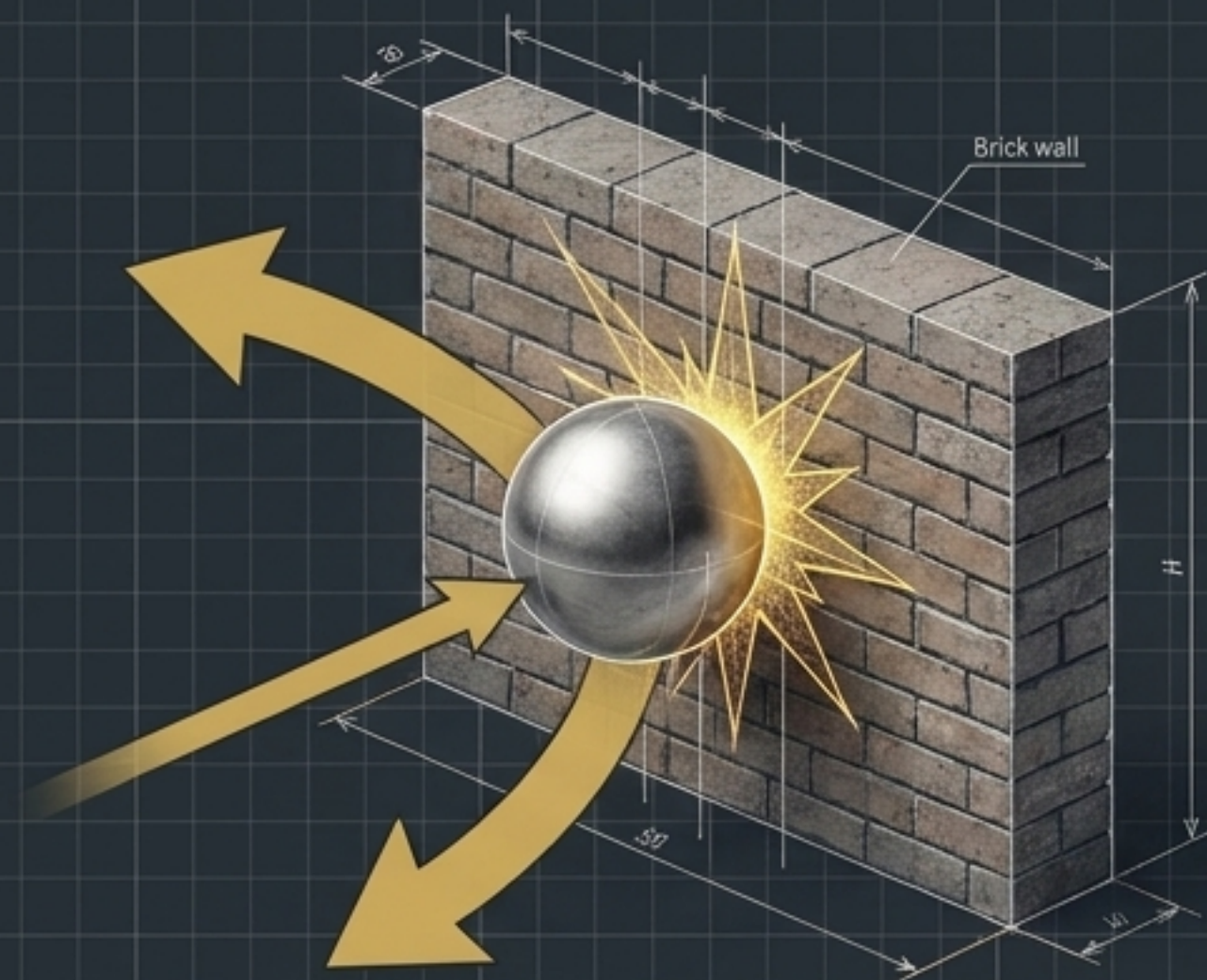


OnCollisionExit

用途: 床から離れた（空中にいる）状
態の判定リセット。

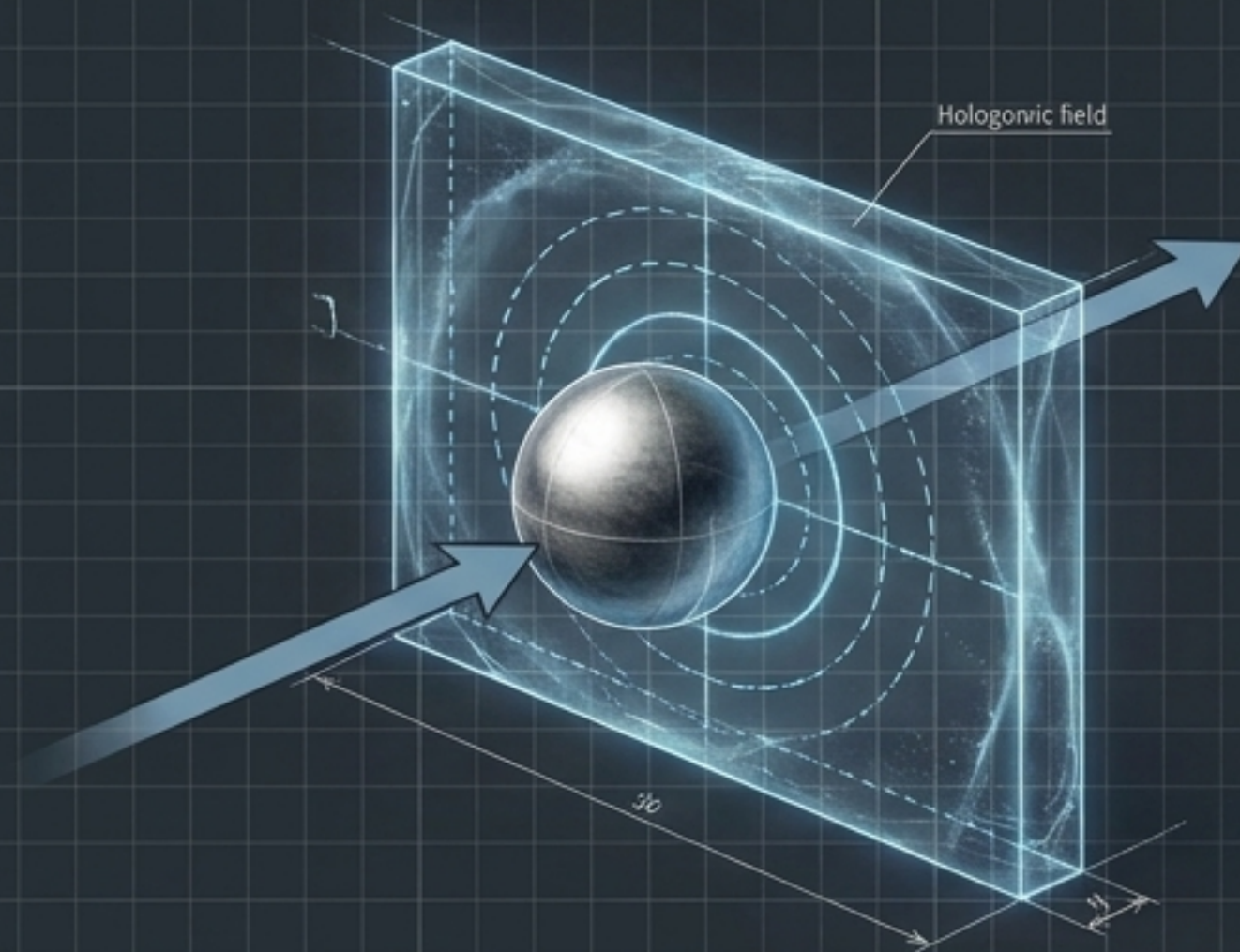
物理的な壁か、見えないセンサーか：Triggerの使い分け

Is Trigger = **OFF**: 物理的な衝突 (Collision)



オブジェクト同士が弾き合う。OnCollisionEnterで検知。

Is Trigger = **ON**: センサー (Trigger)



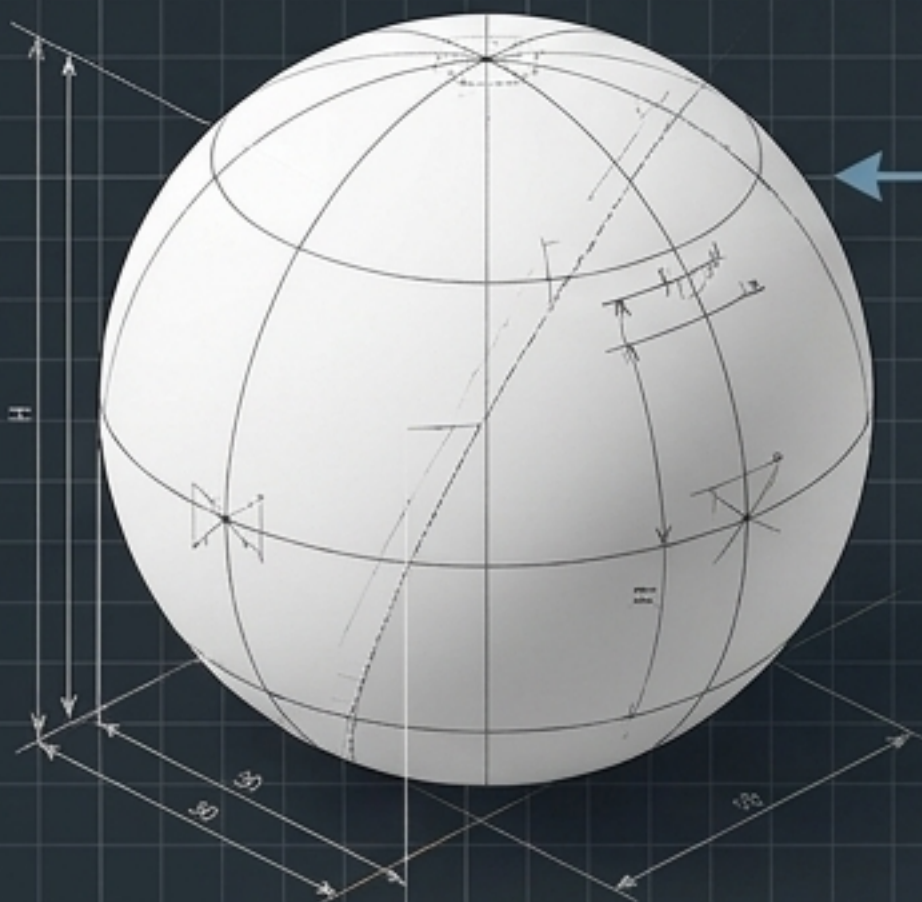
物理的な衝突は起きず、すり抜ける。
アイテムの取得、チェックポイントの通過検知などには
OnTriggerEnter を使用する。

オブジェクトに「質感」を与える：Material & Texture

Base (ワイヤースケーム / 白地)

Albedo (Base Map)

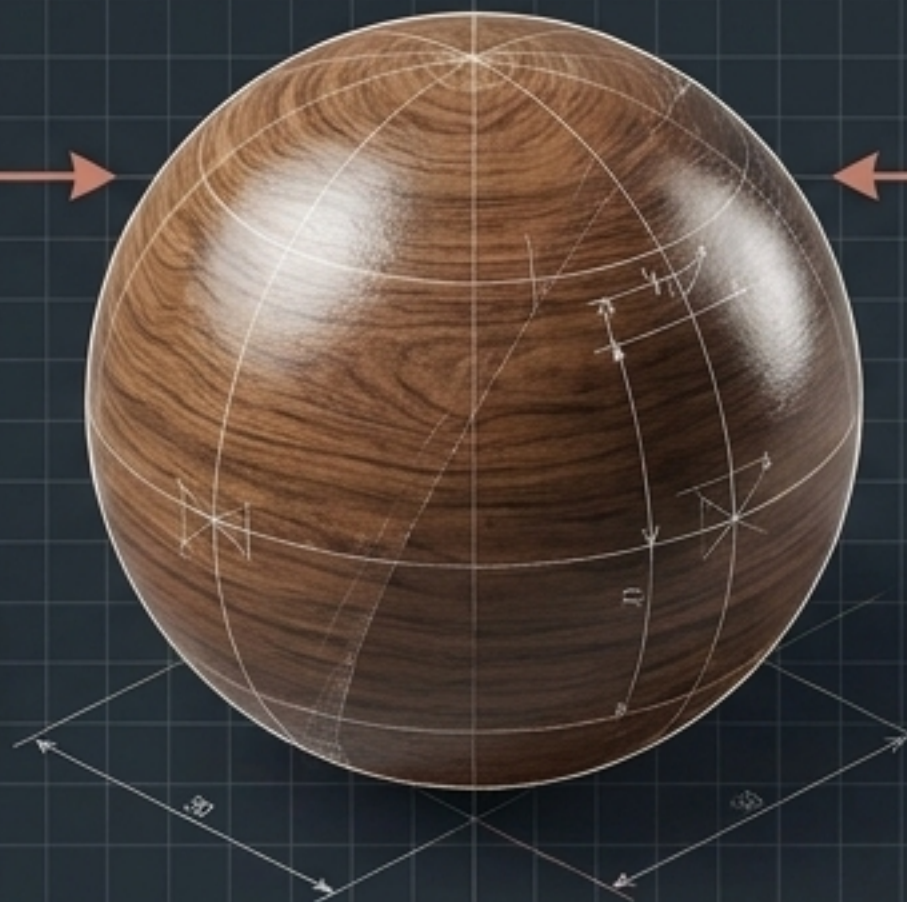
基本となる「色」や、木目・レンガなどの模様 (テクスチャ画像) を設定する。



Surface Reflection (光沢感)

Metallic / Smoothness

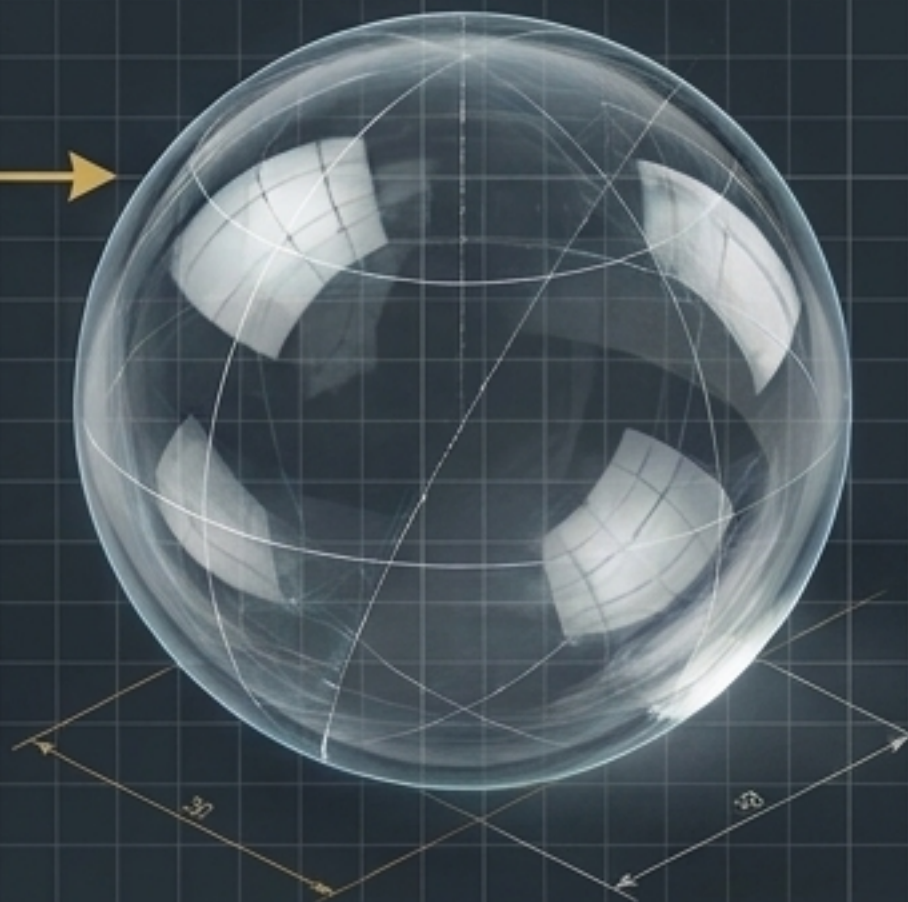
金属のような反射 (Metallic) や、表面のツルツル感・ザラザラ感 (Smoothness) を調整する。



Transparency (透明感)

Rendering Mode -> Transparent

モードを変更し、Albedoのアルファ値を下げることで、ガラスや水のような透ける物体を作る。



Tip: テクスチャがぼやける場合は高解像度画像を使用し、模様の子ズ感ハ Tiling で調整する。

プレイヤーとの対話：TextMeshPro (TMP)

Legacy Text (The Past)



拡大するとぼやける。表現力が乏しい。

TextMeshPro (The Modern Standard)

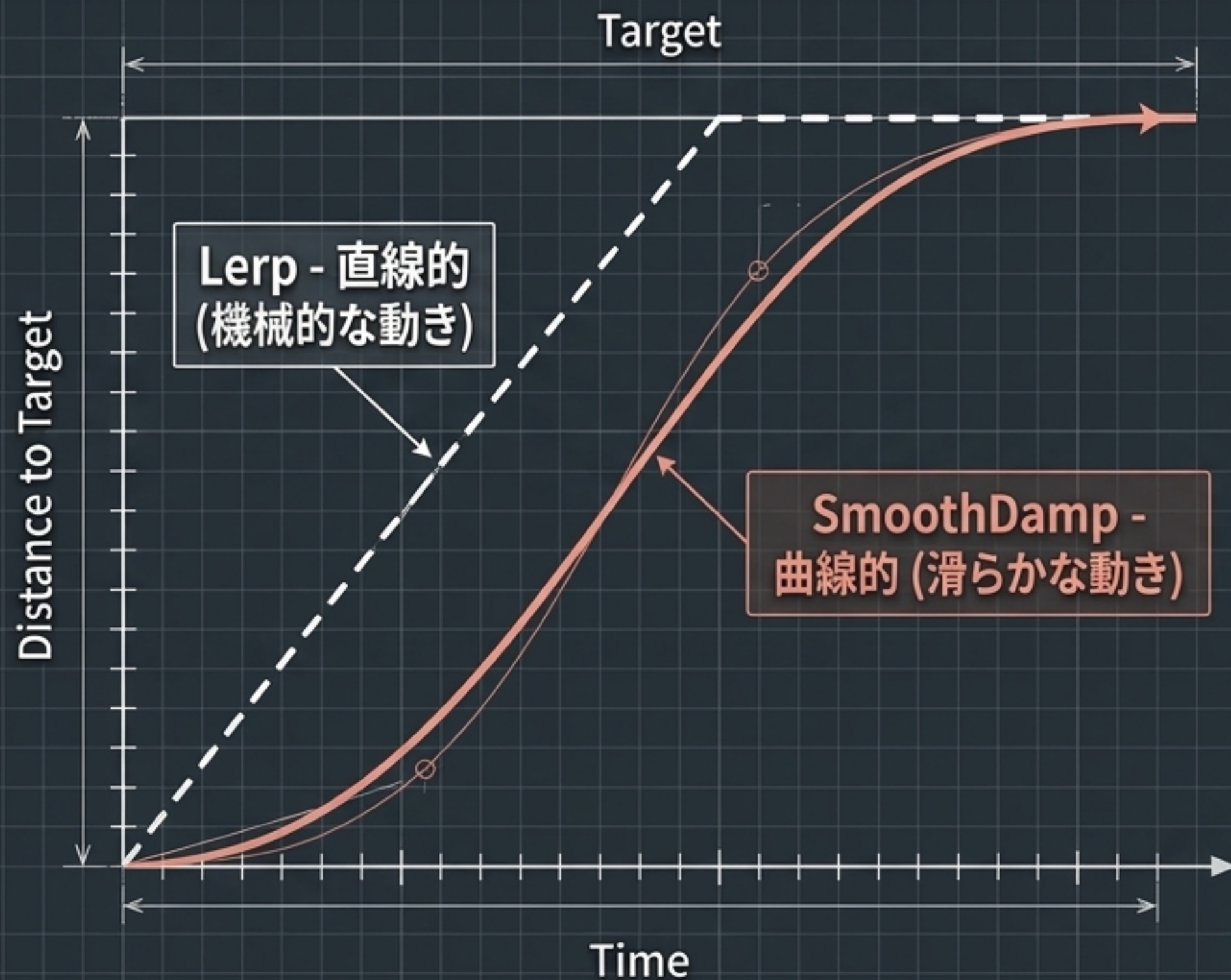


ベクターベースで拡大しても劣化しない。
リッチテキストタグをサポート。

Time: 12<size=50%>min>34.56<size=50%>/size>

スクリプトから操作する際は UnityEngine.UI ではなく **TMPPro** を **using** に追加し、TMP_Text 型の変数でテキストを書き換える。

プロの「手触り」を生み出す魔法：SmoothDamp



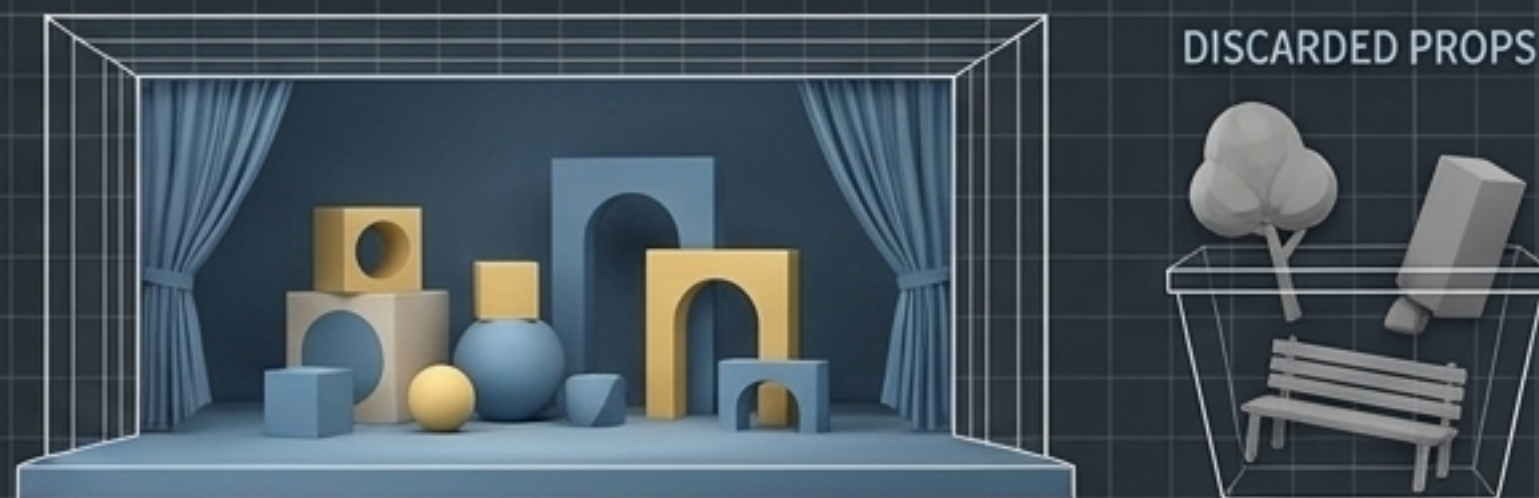
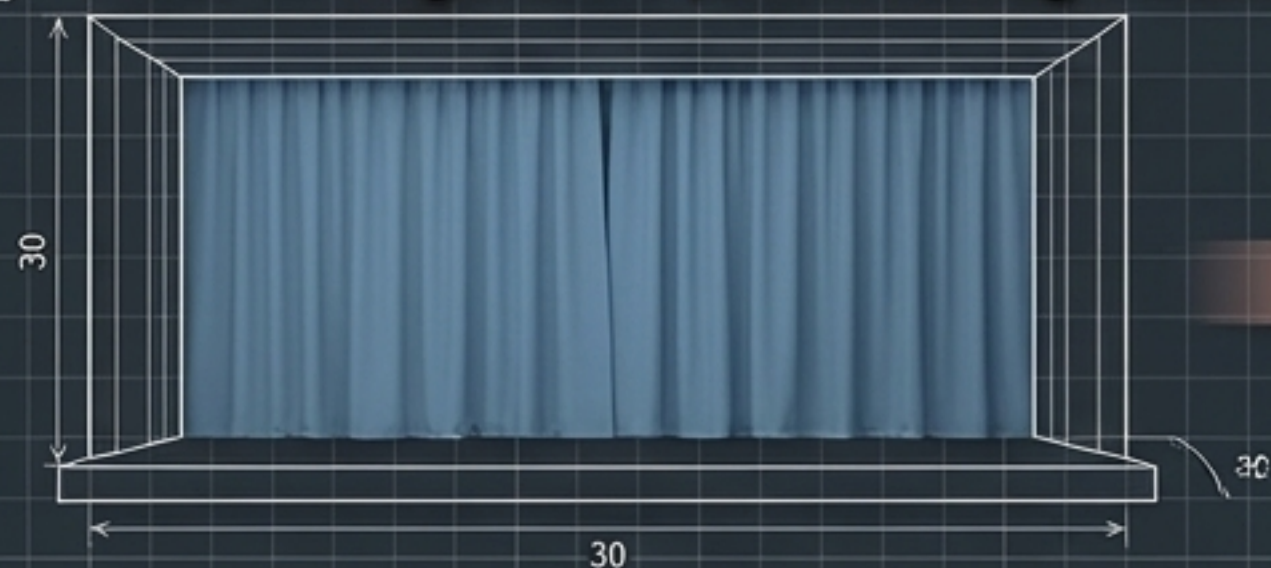
Formula Breakdown

Vector3.SmoothDamp(現在の位置, 目標の位置, **ref** 現在の速度, 到達までの時間, 最大速度)

- **ref velocity**: 内部で自動計算される現在の速度を保持する変数。
- **smoothTime**: 目標に到達するまでのおおよその時間。
- **maxSpeed**: 移動時の速度制限 (省略可能)。

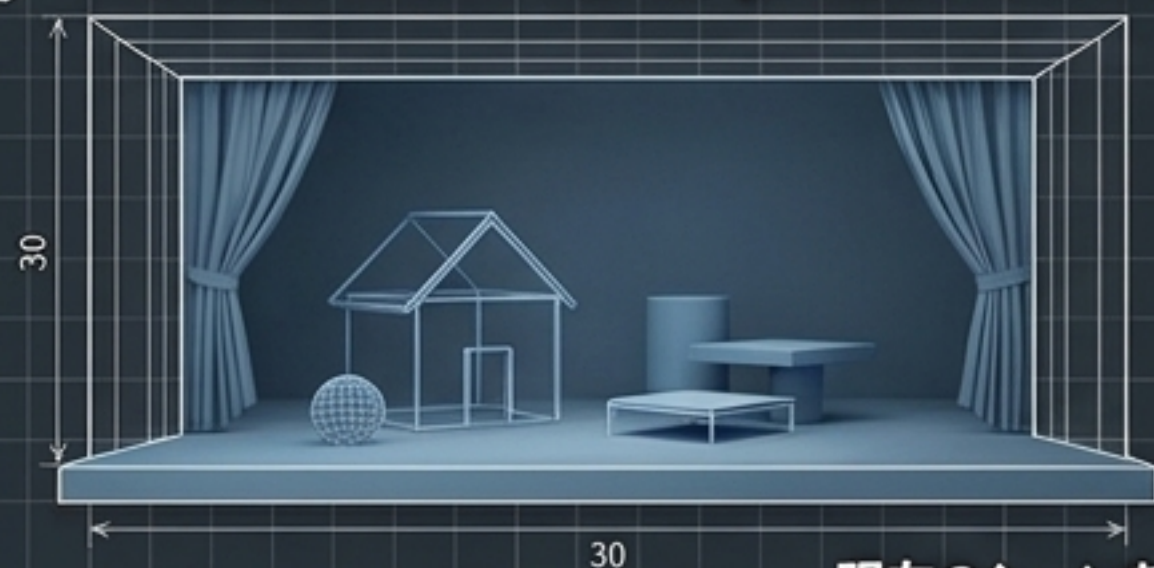
舞台の幕引きと場面転換：SceneManager

Mode 1: Single Load (SceneManager.LoadScene(1))



現在のシーンのオブジェクトはすべて破棄され、新しいシーンをゼロから読み込む。

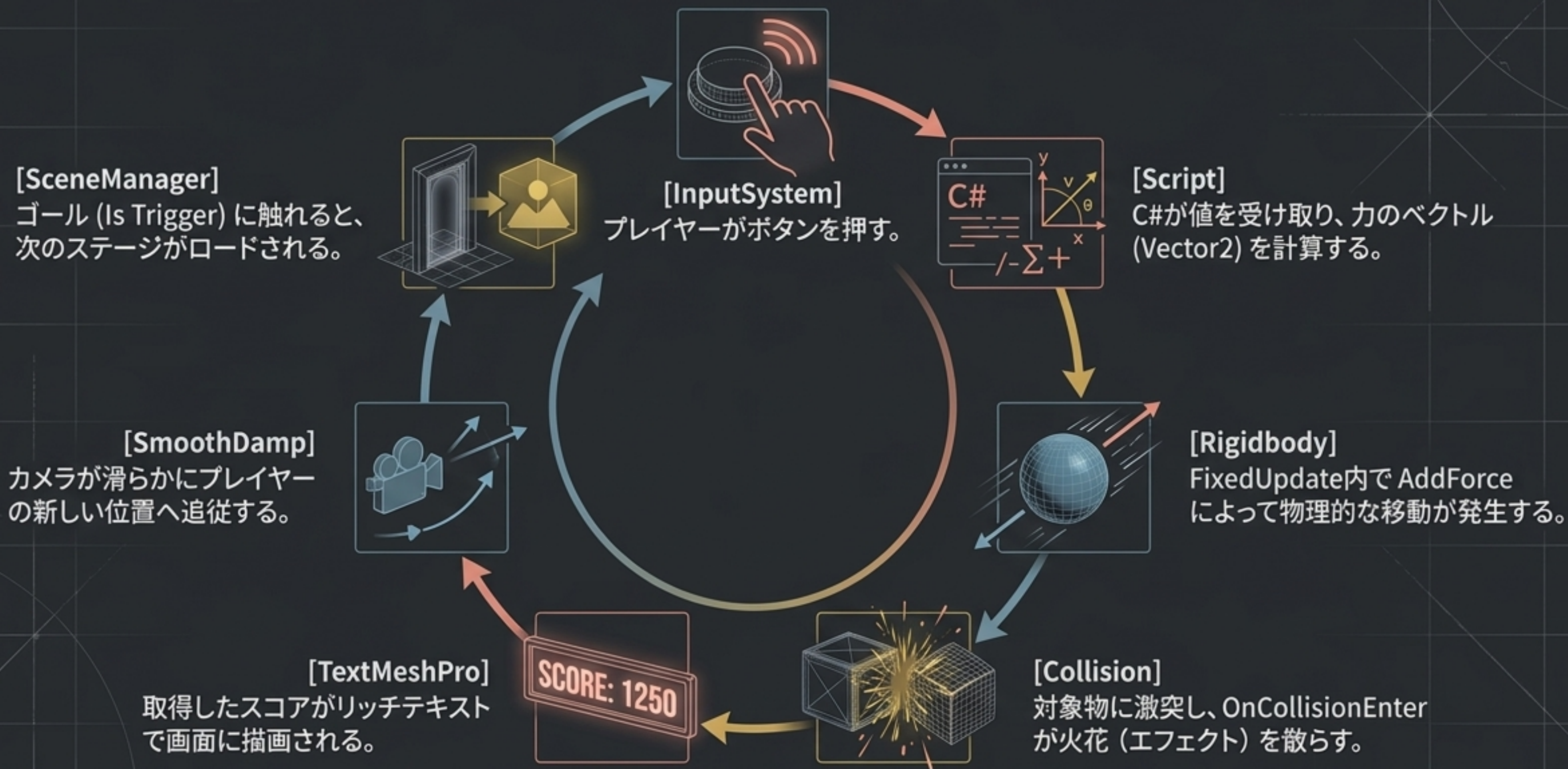
Mode 2: Additive Load (LoadSceneMode.Additive)



現在のシーンを維持したまま、別のシーンを追加で重ねて読み込む。

データ保持の裏技：シーンが切り替わってもスコアやHP (GameManagerなど) を消去したくない場合は、対象のオブジェクトに `DontDestroyOnLoad()` を使用する。

点と点が繋がる瞬間：The Game Loop Complete



設計図から現実へ (From Blueprint to Reality)

Unityは、複雑な魔法の箱ではありません。
「入力」という神経系、「物理演算」という筋肉、「マテリアル」という皮膚。
これらのシンプルなシステムを組み合わせることで、
あなたの想像力はデジタル空間の現実となります。
必要な部品は、すべて揃っています。

